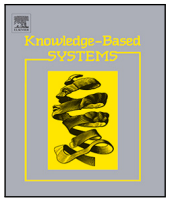
Knowledge-Based Systems 281 (2023) 111068



Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys





HEPM: High-efficiency pattern mining

Xiaojie Zhang [a,b], Guoting Chen [a,*], Linqi Song [b], Wensheng Gan [c], Yunling Song [d]

[a] School of Science, Harbin Institute of Technology (Shenzhen), Shenzhen, China
[b] City University of Hong Kong, Hong Kong, China
[c] College of Cyber Security, Jinan University, Guangzhou, China
[d] Inner Mongolia University, Inner Mongolia, China

ARTICLE INFO

Keywords:
Data mining
High-efficiency pattern
Pruning strategy
Upper bound

ABSTRACT

Pattern mining (PM) is an important field of data mining and has gained considerable momentum recently, mainly owing to the massive growth of big data. PM often sets attentive objectives such as mining frequent or high utility patterns to obtain attractive patterns. High utility patterns address the defect of frequent patterns that cannot reveal the maximum profit. However, it neglects another vital factor, cost or investment. This paper proposes a new high-efficiency PM problem that considers both utility and investment. The problem aims to find patterns with the maximum profit-to-investment ratio. Our paper is devoted to studying high-efficiency itemsets in transaction databases. We first formulate the criteria for a high-efficiency PM problem. Subsequently, we propose a two-phase algorithm called HEPM and an improved one-phase algorithm called HEPMiner to discover high-efficiency patterns in a transaction database. We design a corresponding pruning strategy within HEPM to reduce the search space. In HEPMiner, we utilize a novel efficiency-list and an estimated efficiency co-occurrence structure in the pruning strategies to further improve the mining performance. Moreover, we derive the upper bounds of efficiency for both algorithms. The experimental results demonstrate the effectiveness and efficiency of our two algorithms.

1. Introduction

Pattern mining (PM) [1–4] is an important field of data mining and has received great attention recently, largely owing to the massive growth of big data. PM has been widely used in marketing analysis [5], bioinformatics [6], web finding [7], drug detection [8], and software bug discovery [9]. To support real-world applications, PM aims to optimize practical objectives. For example, the mining of frequent patterns [1,2] and high utility patterns [3,10] are the two most widely studied objectives. To achieve more accurate mining, constraints may also be imposed on the frequent pattern mining problem and the high utility pattern mining problem, such as frequent closed patterns [11], periodic frequent patterns [12,13], and correlated utility patterns [14–16].

Frequent pattern mining (FPM) [1,2] aims to obtain patterns whose frequency is no less than a user-defined minimum threshold value (*minsup*). The user perception is that items that occur frequently are instructive and interesting to groups or individuals. However, FPM fails to achieve optimal benefits because it may miss infrequent but profitable patterns. Accordingly, high utility pattern mining (HUPM) [3,10] is designed to remedy this limitation. HUPM addresses the challenge of FPM intending to find maximized utility patterns, not only considering if they are frequent. HUPM defines *utility* as the total products of item quantity and item profit. HUPM aims to obtain patterns whose *utility* is no less than a user-defined minimum utility threshold value (*minutil*) [3].

Regarding HUPM, most research ignores a valuable factor: investment or cost. Investment means a commitment in money or time now to generate benefits (utility) in the future. It is important to note that while investment can lead to high profits or benefits, it is not a guarantee. Some investments may not yield the expected returns, and there may be a risk of losing money. Therefore, it is crucial to carefully weigh the potential benefits and risks before making decisions. In the financial field, annual return on investment is a useful metric when examining the balance between profitability and risk [17]. It provides insight into how much return an investment generates over a given period relative to the initial cost and helps to determine if the investment is worthwhile. We incorporate the main point of this measurement into the PM field to define a new metric called *efficiency*. This metric helps us better understand the trade-off between the potential for profit and the potential for risk from statistical data. It contributes to obtaining

* Corresponding author.
E-mail addresses: xzhang3338-c@my.cityu.edu.hk (X. Zhang), chenguoting@hit.edu.cn (G. Chen), linqi.song@cityu.edu.hk (L. Song), wsgan001@gmail.com (W. Gan), syunling@163.com (Y. Song).

https://doi.org/10.1016/j.knosys.2023.111068
Received 30 April 2023; Received in revised form 29 September 2023; Accepted 12 October 2023
Available online 14 October 2023
0950-7051/© 2023 Elsevier B.V. All rights reserved.

Table 1
Top 10 HUIs *versus* top 10 HEIs.

HUIs: utility	HEIs: *efficiency*
$\{b, c, d, e\}$: 41	$\{a\}$: 0.45
$\{b, c, e\}$: 38	$\{b\}$: 0.33
$\{b, d, e\}$: 36	$\{d\}$: 0.29
$\{b, c, d\}$: 35	$\{b, d\}$: 0.26
$\{b, c, e, f\}$: 34	$\{a, c\}$: 0.25
$\{a, c, e\}$: 32	$\{b, c\}$: 0.21
$\{b, e\}$: 31	$\{c\}$: 0.2
$\{a, b, c, d, e, f\}$: 31	$\{a, d\}$: 0.19
$\{a, c\}$: 30	$\{b, e\}$: 0.186
$\{b, d\}$: 30	$\{b, c, d\}$: 0.184

the most investment-efficient patterns by considering both utility and investment.

The first naive idea would be to directly use *efficiency* instead of utility in classical utility mining algorithms. However, since the utility of an item is the product of unit profit and quantity, it does not make sense to replace unit profit with unit efficiency in this definition. Hence, we need to find a way to handle this situation. To illustrate the difference between *utility* and *efficiency*, let us consider an example. There are two types of patterns: {apple, banana} and {durian, cherry}. Suppose we invest 1000$ for {apple, banana} and 10000$ for {durian, cherry}; then, we obtain utility 1000$ for {apple, banana} and 9000$ for {durian, cherry}. If we only focus on the utility constraint, {durian, cherry} is more impactful since they have a higher utility (9000$). However, if we consider both utility and investment, we find that we invest 1000$ for a return of 1000$ in the first case and invest 10000$ for a return of 9000$ in the second case. The first pattern is more impactful since it has a higher ratio of utility to investment (1.0) than the other (0.9). In the above case, although the utility for {durian, cherry} is higher, we are inclined to choose {apple, banana}. Since {apple, banana} seems more investment-efficient when considering the trade-off between profitability and risk. This means that {apple, banana} is more worthwhile to invest than {durian, cherry}.

For comparison, Table 1 shows the top 10 high-utility itemsets (HUIs) and high-efficiency itemsets (HEIs) of the transaction database in Table 2. The inconsistent high utility and efficiency patterns can be attributed to ignoring investment in the HUPM structure. Simply focusing on utility may not provide an accurate picture of the *efficiency* of a particular pattern. Therefore, when analyzing patterns and making decisions, it is essential to consider both the utility and the investment. Doing so can help decision-makers identify efficient patterns that provide the best return on investment and make more informed and effective decisions.

To address this issue, we formulate the high-efficiency pattern mining (HEPM) problem. HEPM considers not only the utility but also the investment in order to obtain the optimal ratio of utility to investment. In a traditional FPM problem, the downward closure property of support helps reduce the search space and obtain the final frequent itemset quickly. However, in HEPM, the measurement *efficiency* does not satisfy the downward closure property. To solve this problem, we derive upper bounds related to measurement *efficiency* that satisfy the downward closure property to help reduce the search space. To summarize, we make the following main contributions.

- We propose a new measurement *efficiency* to find the most investment-efficient patterns (also high-efficiency patterns) by considering both utility and investment.
- We propose a new upper bound *EUB* that satisfies the downward closure property, and we use it to reduce the search space in a two-phase algorithm called HEPM.
- We propose an improved one-phase algorithm called HEPMiner and four corresponding novel pruning strategies. The efficiency-list and estimated efficiency co-occurrence structure (EECS) are proposed to improve the mining performance.
- To show the effectiveness and efficiency of the proposed algorithms, we perform extensive experiments using different pruning strategies under distinct hyperparameters in seven datasets. We show that the proposed algorithms can effectively mine high-efficiency patterns, while HUPM may miss those important patterns. Both algorithms can find accurate high-efficiency patterns (HEPs) and show good scalability.

The remainder of the paper is organized as follows. In Section 2, some research studies about frequent mining versus high-utility mining are introduced. Section 3 introduces some basic knowledge about HEPM. In addition, we define the problem model of HEPM and its corresponding standards. In Section 4, we propose an upper bound *EUB* and the corresponding pruning strategy. Details of the HEPM algorithm utilizing *EUB* and the pruning strategy are also presented. In Section 5, we introduce new upper bounds *SEUB* and *SEUBW*, several novel pruning strategies, and new structures (efficiency-list and EECS). Then we utilize them to present the HEPMiner algorithm. In Section 6, we describe the experimental process, experimental results, and result analysis of HEPM and HEPMiner. Finally, Section 7 provides the conclusions.

2. Related work

2.1. Frequent pattern mining

In the last few decades, frequent pattern mining has been extensively studied. Frequent patterns mainly include itemset [2], association rule [1], episode [18], sequence [19], and so on. For itemset mining or association rule mining, typical examples of algorithms are Apriori [1], Eclat [20], and FP-growth [2]. The Apriori algorithm [1] utilizes the downward closure principle (if an itemset is frequent, all its subsets must be frequent) to reduce the search space. However, it needs to scan the database multiple times. To solve this problem, the Eclat algorithm [20] employs vertical data representation to reduce the time of transaction database scans. In addition, the FP-growth algorithm [2] scans the database only twice, and it is several orders of magnitude faster than Apriori and Eclat. FP-growth works based on a compact data structure called the FP-tree. Some extensions depending on frequent itemset mining are also full of obstacles, such as periodic frequent itemset mining [21,22], closed frequent itemset mining [11,23,24], skyline frequent-based itemset mining [25], and privacy-protecting frequent itemset mining [26–28].

2.2. High-utility pattern mining

Utility-driven pattern mining is different from frequency-based pattern mining. In general, utility theory comes from economics, and utility-driven mining assimilates utility theory into data mining. These methods focus on finding results, maximizing overall benefit rather than frequency. High utility itemset mining (HUIM) [3,10,29] has been proposed and has attracted much attention. In response to several active issues, high-utility occupancy itemset mining [30], top k HUIM [31], and incremental HUIM [32] were developed. Others, such as on-shelf HUIM [33], HUIM with negative items [34,35], and targeted utility mining [36,37] also began to be developed. The initial version of HUIM is a two-phase algorithm [3]. It generates high transaction weight utility itemsets (HTWUIs) in the first stage and discovers high utility itemsets (HUIs) in the second stage. UP-growth [38] invents a UP-tree structure and new upper bounds. This algorithm only executes two database scans. The HUI-Miner algorithm [10] crosses the gap from two phases to one phase without generating candidates.

Table 2
Transaction database D.

tid	Transaction	TU
1	$(a,1)$, $(b,5)$, $(c,2)$, $(d,3)$, $(e,1)$, $(f,1)$	31
2	$(b,4)$, $(c,3)$, $(d,3)$, $(e,1)$	20
3	$(a,1)$, $(c,2)$, $(d,1)$	9
4	$(a,2)$, $(c,6)$, $(e,2)$, $(g,5)$	27
5	$(b,2)$, $(c,2)$, $(e,1)$, $(f,1)$, $(g,2)$	16

Table 3
External utility of items.

item	a	b	c	d	e	f	g
unit profit	5	2	1	2	3	5	1

It also initially proposes a utility-based list structure. FHM [29] further optimizes HUI-Miner by reducing the number of combinations of utility-lists to increase performance. Some extensions depend on high utility itemset mining, such as periodic high utility itemset mining [39,40], closed high utility itemset mining [41,42], skyline utility-based itemset mining [25], and correlated high utility itemset mining [14–16]. Traditional FPM and HUPM do not consider investment or cost factors. Fournier-Viger et al. [43] first proposed the concept of cost-effective patterns based on sequence mining [44–46], in which they also considered a cost indicator. However, the application scene is only suitable for sequence databases with fixed sequence utility but not for transaction data.

3. Preliminaries and problem statement

This section defines our problem and high-efficiency pattern mining standards for transaction databases.

Definition 1 (*Transaction Databases*)**.** A transaction database is a set of transaction records. Every transaction record has a unique tag-mark *tid*, such as T_1 or 1. A transaction record is a set of items from I with the corresponding quantities. Each item $i \in I$ is associated with a positive number $p(i)$, called its external utility. Additionally, every item i that appears in a transaction T_p has a positive number $q(i,T_p)$, called its internal utility, which represents the purchased quantity of that item in the transaction.

This context is often used in market basket analysis, where the external utility represents the unit profit of an item (e.g., the profit from selling one unit of bread), and the internal utility represents the quantity purchased (e.g., the number of units of bread bought in a transaction). For example, in Table 2, the transaction database $D = \{T_1, T_2, T_3, T_4, T_5\}$. For $T_3 = \{(a,1), (c,2), (d,1)\}$, the included items are a, c, and d with their corresponding quantities: 1, 2, and 1. This indicates that in transaction T_3, a person bought one unit of commodity a, one unit of commodity d, and two units of commodity c. Table 3 lists the unit profit of different items.

Definition 2 (*Investment*)**.** Investment in certain types of items shows a variety of forms, including time, energy, money, and others. The unit investment of an item i is represented by $ui(i)$. The investment of an item i in a single transaction T_p is defined as $inv(i,T_p) = ui(i) * q(i,T_p)$, which is the product of unit investment and quantity. The total investment of item i in database D is defined as $inv(i) = \sum_{i \in T_p} inv(i,T_p)$, which is the accumulation of the investment in all transactions containing i. The investment of an itemset E in a database D is the accumulation of investment of all items in E. This is denoted as $inv(E) = \sum_{i \in E} inv(i)$.

Table 4
Investment of items.

item	a	b	c	d	e	f	g
unit investment	11	6	5	7	20	50	40
total investment	44	66	75	49	100	100	80

For example, the second row of Table 4 represents the unit investments of different items i in the database D in Table 2. The total investment of different items can be computed from the database and is shown in the third row. For instance, the unit investment of item c is $ui(c) = 5$. The investment of item c in T_1 is $inv(a,T_1) = 5 * 2 = 10$. The investment of item c in D is $inv(c) = inv(c,T_1) + inv(c,T_2) + inv(c,T_3) + inv(c,T_4) + inv(c,T_5) = 10 + 15 + 10 + 30 + 10 = 75$. The investment of itemset $\{c,e\}$ is $inv(\{c,e\}) = inv(c) + inv(e) = 75 + 100 = 175$.

Definition 3 (*Utility*)**.** The utility of an item i in a single transaction T_p is defined as $u(i,T_p) = p(i) * q(i,T_p)$, which is the product of external utility and internal utility. The utility of an itemset E in one transaction T_p is the accumulation of the utility of all items in E. This can be denoted as $u(E,T_p) = \sum_{i \in E} u(i,T_p)$. The utility of an itemset E in a database D is the accumulation of the utility of the itemset E in every transaction containing E. This can be denoted as $u(E) = \sum_{T_p \in D \wedge E \subset T_p} u(E,T_p)$.

For example, the utility of item a in transaction T_4 is $u(a,T_4) = 5 \times 2 = 10$. Suppose itemset $E = \{c,e\}$, the utility of E in transaction T_1 is $u(\{c,e\},T_1) = 1 \times 2 + 3 \times 1 = 5$. The utility of E in D is $u(\{c,e\}) = u(\{c,e\},T_1) + u(\{c,e\},T_2) + u(\{c,e\},T_4) + u(\{c,e\},T_5) = 5 + 6 + 12 + 5 = 28$.

Definition 4 (*HUIs*)**.** Suppose the minimum utility threshold value is η. If the utility of an itemset Q in D is no less than η, then Q is a high utility itemset (HUI).

For example, suppose $\eta = 10$, $u(\{c,e\}) = 28 \geqslant \eta$, $\{c,e\}$ is an HUI.

Definition 5 (*Efficiency*)**.** *Efficiency* represents utility on investment. The *efficiency* of an item i in a transaction T_p is $e(i,T_p) = \frac{u(i,T_p)}{inv(i)}$. The *efficiency* of an itemset E in a transaction T_p is $e(E,T_p) = \frac{\sum_{i \in E} u(i,T_p)}{\sum_{i \in E} inv(i)}$. The *efficiency* of an itemset E in database D is $e(E) = \frac{u(E)}{inv(E)} = \frac{\sum_{T_p \in D \wedge E \subset T_p} u(E,T_p)}{inv(E)} = \sum_{T_p \in D \wedge E \subset T_p} e(E,T_p)$.

For example, the *efficiency* of item a in transaction T_4 is $e(a,T_4) = (5 \times 2)/44 \approx 0.23$. If the itemset $E = \{c,e\}$, then the *efficiency* of E in transaction T_1 is $e(E,T_1) = (1 \times 2 + 3 \times 1)/(75 + 100) \approx 0.029$. The *efficiency* of E in D is $e(E) = \frac{\sum_{E \subset T_p} u(E,T_p)}{inv(E)} = [\sum_{i \in E} u(i,T_1) + \sum_{i \in E} u(i,T_2) + \sum_{i \in E} u(i,T_4) + \sum_{i \in E} u(i,T_5)]/inv(E) = (5 + 6 + 12 + 5)/(75 + 100) = 0.16$.

Definition 6 (*HEIs*)**.** Given transaction database D and minimum efficiency threshold value γ, if the *efficiency* $e(E)$ of an itemset E in D is no less than γ, then E is a high-efficiency itemset (HEI).

For example, suppose $\gamma = 0.2$, since the *efficiency* of $E = \{c,e\}$ is $e(E) = 0.16 < \gamma$, then E is not an HEI.

Problem statement: Given transaction database D and minimum efficiency threshold value γ, the problem of high-efficiency pattern mining (HEPM) is to find high-efficiency itemset P that meets $e(P) \geqslant \gamma$. This paper aims to design algorithms to solve the above problem.

In this paper, we propose two methods to solve this problem. Their frameworks are shown in Fig. 1.

4. HEPM algorithm

This section mainly describes the search tree space, the upper bound with the corresponding pruning strategy, and the HEPM algorithm to find HEIs.

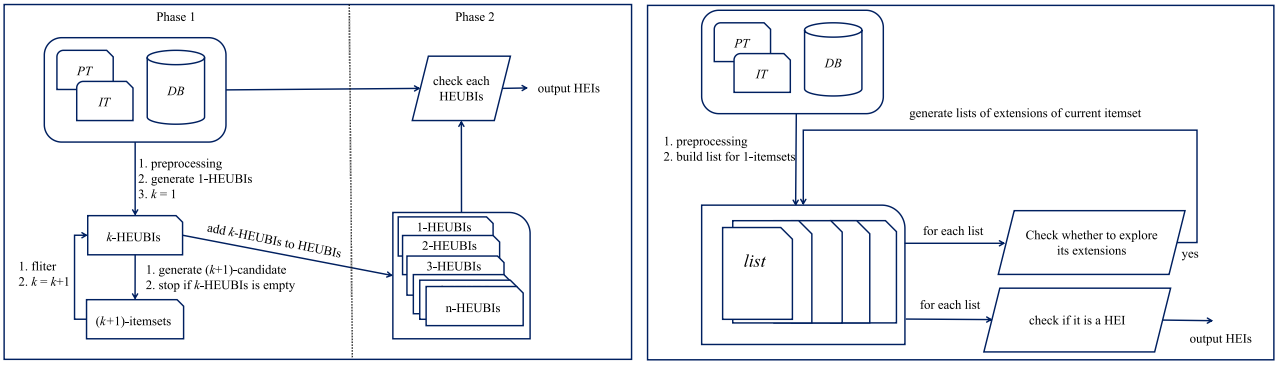


Fig. 1. Framework of HEPM and HEPMiner respectively.

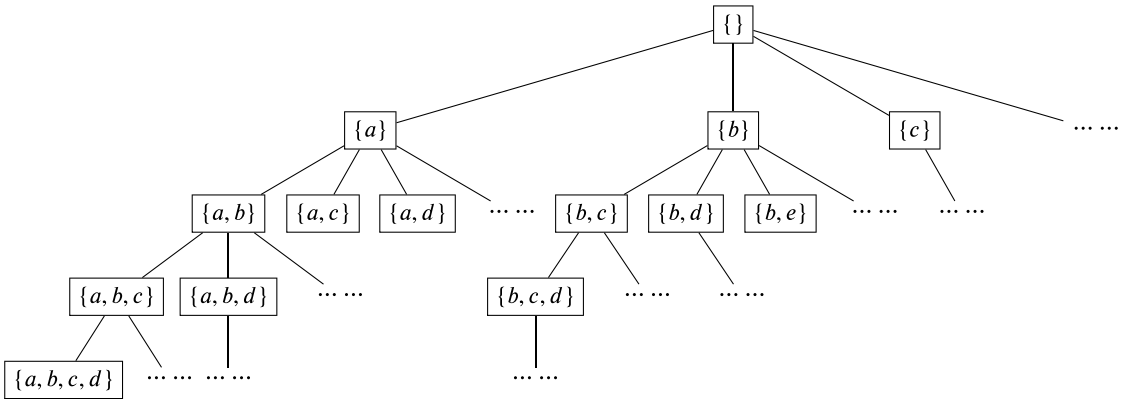


Fig. 2. A lexicographic tree for the running example.

4.1. Lexicographic tree

To extract HEIs from a transaction database, a large search space must be explored. This search space is a lexicographic tree, as shown in Fig. 2. In a lexicographic tree, each node represents an itemset that needs to be checked, and each branch represents an extension operation from the current node. The search process begins at the root node, representing the empty set $\{\}$. The child nodes of the root node, such as $\{a\}$ and $\{b\}$, represent itemsets that can be extended from the empty set. The items are arranged in a predefined sequence to avoid visiting and evaluating repeated itemsets, such as $\{a, b\}$ and $\{b, a\}$. Fig. 2 illustrates the search space in lexicographical order, but other orders can also be used. Due to space limitations, the lexicographic tree in the figure is incomplete.

To mine HEIs, each node representing an itemset must be checked to determine if it is an HEI. However, checking all nodes in a tree is time-consuming, so pruning strategies are necessary to limit the search space and save time. In the following section, we describe some new pruning strategies.

4.2. Upper bound and pruning strategy

In Definition 5, the denominator of *efficiency* $e(E)$ is an investment that satisfies monotonicity, but the numerator is utility, which does not satisfy monotonicity or antimonotonicity. Therefore, $e(E)$ does not satisfy monotonicity or antimonotonicity. We can also see it from an example. Consider Table 1, where the second column shows the top 10 high-efficiency itemsets. The *efficiency* of $\{c\}$ is 0.2, while the *efficiency* of $\{a, c\}$ is 0.25, indicating that the *efficiency* of an extension itemset is higher than that of the original itemset. Conversely, the *efficiency* of $\{b, c, d\}$ is 0.184, indicating that the *efficiency* of an extension is lower than the original itemset.

To reduce the search space, we propose a substitute that meets the antimonotonicity to filter redundant nodes. This substitute is called the efficiency upper bound (*EUB*).

Definition 7 (*TU and TWU*)**.** The transaction utility (*TU*) is the accumulation of the utility of each item in a transaction T_p which can be denoted as $TU(T_p) = \sum_{i \in T_p} u(i, T_p)$. The *TWU* of an itemset E is the accumulation of *TU* of transactions that contain the itemset E. This can be denoted as $TWU(\mathrm{E}) = \sum_{E \subseteq T_p} TU(T_p)$.

For example, $TU(T_3) = u(c, T_3) + u(a, T_3) + u(d, T_3) = 2 + 5 + 3 = 9$. *TU* values of other transactions are computed in Table 2. $TWU(\{c, e\}) = TU(T_1) + TU(T_2) + TU(T_4) + TU(T_5) = 31 + 20 + 27 + 16 = 94$.

Definition 8 (*Efficiency Upper Bound (EUB)*)**.** The efficiency upper bound (*EUB*) of an itemset E equals $TWU(E)$ divided by $inv(E)$, which can be denoted as $EUB(E) = TWU(E)/inv(E)$.

Theorem 1. *Given any itemset X and its extension X', we have the following inequalities: $e(X') \leqslant EUB(X') \leqslant EUB(X)$.*

Proof. Suppose that X is a certain itemset and X' is an extension itemset of X. We know that *TWU* is an upper bound of utility and that *TWU* meets antimonotonicity, so $u(X') \leqslant TWU(X') \leqslant TWU(X)$. Then $e(X') = u(X')/inv(X') \leqslant TWU(X')/inv(X') = EUB(X') \leqslant TWU(X)/inv(X')$. From the monotonicity of investment, we know that $inv(X) \leqslant inv(X')$. Therefore, $e(X') \leqslant EUB(X') \leqslant TWU(X)/inv(X') \leqslant TWU(X)/inv(X) = EUB(X)$.

Definition 9 (*High EUB Itemsets*)**.** If an itemset X satisfies $EUB(X) \geqslant \gamma$, then we call X a high EUB itemset (HEUBI).

From Theorem 1, we can prove the following result which will be implemented in a pruning strategy.

Theorem 2. *Given any itemset X and its extension X', if $EUB(X) < \gamma$, then $EUB(X') < \gamma$. That is, if X is not a HEUBI, then X' is also not a HEUBI.*

Proof. Suppose that X is certain itemset and X' is an extension itemset of X; we obtain $e(X') \leqslant EUB(X') \leqslant EUB(X)$. If $EUB(X) < \gamma$, it is obviously $e(X') \leqslant EUB(X') < \gamma$. It also means that if X is not a HEUBI, then X' is also not a HEUBI.

For example, according to the transaction database in Tables 2 to 4, we obtain $EUB(\{a,c\}) = TWU(\{a,c\})/inv(\{a,c\}) = (31+9+27)/(44+75) = 67/119 \approx 0.56$. If $\gamma = 0.6$, $\{a,c\}$ is not a HEUBI, and any extensions of $\{a,c\}$ are also not HEUBIs.

We have the following corollary from Theorem 2 and its proof.

Corollary 1. *HEIs are always HEUBIs, but HEUBIs are not always HEIs.*

Pruning strategy. For any itemset X, if $EUB(X) < \gamma$, then there is no need to explore X and any of its extensions X'.

This pruning strategy will help the mining method to filter out many nonvaluable nodes in the lexicographic tree. For example, consider $\gamma = 0.6$, and the mining process is now visiting the node $\{a,c\}$. The mining process does not need to visit descendants of the node $\{a,c\}$, since $EUB(\{a,c\}) \approx 0.56 < \gamma$.

4.3. Algorithm

In this section, we mainly illustrate the details of the HEPM algorithm. HEPM is a two-phase method. In phase one, it obtains HEUBIs according to the closure property *EUB*. In phase two, it filters out HEIs from HEUBIs by computing the *efficiency* value. Algorithm 1 is the main algorithm, which calls for Algorithm 2 and Algorithm 3 (Lines 1 and 2). Algorithm 2 is a phase-one process, and Algorithm 3 is a phase two process.

Algorithm 1: HEPM

Input: D, a transaction database; PT, a profit table; IT, an investment table; γ, a minimum efficiency threshold.
Output: *HEIs*, the complete set of HEIs.

1 $HEUBIs$ = gen_HEUBIs(D, PT, IT, γ);
2 $HEIs$ = gen_HEIs(D, PT, IT, γ, $HEUBIs$);
3 **return** *HEIs*

Algorithm 2: gen_HEUBIs

Input: D, a transaction database; PT, a profit table; IT, an investment table; γ, a minimum efficiency threshold.
Output: *HEUBIs*, the complete set of HEUBIs.

1 scan D to:
2 (a) calculate investment inv of each items in D;
3 (b) calculate EUB of each 1-itemset in D;
4 (c) put 1-itemsets which satisfy $EUB(X) \geqslant \gamma$ in C_1;
5 (d) sort 1-itemsets in C_1 lexicographically;
6 $HEUBIs = C_1$;
7 $k = 1$;
8 **while** $C_k \neq \emptyset$ **do**
9 C_{k+1} = apriori_gen(C_k);
10 calculate EUB of each (k+1)-itemset in C_{k+1};
11 filter (k+1)-itemsets in C_{k+1} with $EUB(X) < \gamma$;
12 $HEUBIs = HEUBIs \cup C_{k+1}$;
13 $k = k + 1$;
14 **end**
15 **return** *HEUBIs*

In Algorithm 2, the input includes a transaction database D, the profit table PT, the investment table IT, and a minimum efficiency threshold γ. The output is HEUBIs. The algorithm first obtains alphabetically sorted high-efficiency upper bound 1-itemsets in Lines 1 to 5. Then it iteratively generates HEUBIs of 2-itemsets depending on 1-itemsets, HEUBIs of 3-itemsets depending on 2-itemsets, and so on until no new HEUBIs are generated. In every iteration, apriori_gen() generates suspected HEUBIs of (k+1)-itemsets according to HEUBIs of k-itemsets. It first ensures that HEUBIs of k-itemsets are sorted alphabetically, and every k-itemset only needs to be combined with the latter k-itemsets. To combine two k-itemsets, they must have identical previous $(k-1)$ items. Then the new (k+1)-itemset is the identical $(k-1)$ items appended to the last items of the two k-itemsets. The newly generated suspected HEUBIs of $(k+1)$-itemsets must be filtered according to the pruning strategy related to *EUB*. In the end, it returns the complete set of HEUBIs.

In Algorithm 3, the input has an extra parameter *HEUBIs* which is the output of Algorithm 2. The output of Algorithm 3 is HEIs. The algorithm visits every itemset of HEUBIs and computes its actual *efficiency* by scanning D. The itemsets satisfying efficiency threshold conditions are kept in HEIs and finally returned.

Algorithm 3: gen_HEIs

Input: D, a transaction database; PT, table; IT, investment table; γ, a minimum efficiency threshold; *HEUBIs*, the set of HEUBIs.
Output: *HEIs*, the complete set of HEIs.

1 $HEIs = \emptyset$
2 **for** $c \in HEUBIs$ **do**
3 scan D to compute the *efficiency* of c
4 **if** $e(c) \geqslant \gamma$ **then**
5 $HEIs = HEIs \cup c$
6 **end**
7 **end**
8 **return** *HEIs*

5. HEPMiner algorithm

The previous HEPM algorithm utilizes the Apriori structure in phase one, which computes *EUB* for suspected HEUBIs. In addition, it also conducts multiple database scans to calculate the real *efficiency* values of HEUBIs in phase two. These time-consuming processes motivate us to design an improved algorithm for reducing database scans and obtaining results in one phase. Details of the algorithm HEPMiner are presented below.

5.1. Efficiency-list structure

To improve the performance of mining HEIs, a new structure, called the efficiency-list, is created to store intermediate values.

Definition 10 (*Remaining and Internal Efficiency*)**.** Let E be an itemset in T_p; the remaining itemset of E in T_p is the set of all items in $T_p - E$ that occur after every item of E, which we denote as $T_p \backslash E$. The remaining utility of E in T_p is defined as $ru(E,T_p) = \sum_{i \in T_p \backslash E} u(i,T_p)$. The remaining efficiency value of E in T_p is defined as $re(E,T_p) = \frac{ru(E,T_p)}{inv(E)} = \frac{\sum_{i \in T_p \backslash E} u(i,T_p)}{inv(E)}$. The sum of the remaining efficiency of E in a database D is the accumulation of all remaining efficiency values of E in each transaction containing E. This can be denoted as $sumRe(E) = \sum_{E \subset T_p} re(E,T_p)$. To distinguish it from the previous concept, we call the previously defined *efficiency* the internal efficiency, which is $ie(E,T_p) = e(E,T_p)$. Then, $sumIe(E)$ denotes the sum of the internal efficiency of itemset E in the database D, $sumIe(E) = \sum_{E \subset T_p} ie(E,T_p)$. Specifically, $sumIe(E)$ is equivalent to $e(E)$.

{a}		
tid	***ie***	***re***
1	0.1136	0.5909
3	0.1136	0.0909
4	0.2272	0.3864
inv: 44	***sumIe***: 0.4545	***sumRe***: 1.0682

{b}		
tid	***ie***	***re***
1	0.1515	0.2424
2	0.1212	0.1818
5	0.0606	0.1818
inv: 66	***sumIe***: 0.3333	***sumRe***: 0.6061

{c}		
tid	***ie***	***re***
1	0.0267	0.1867
2	0.04	0.12
3	0.0267	0.0267
4	0.08	0.1467
5	0.0267	0.1333
inv: 75	***sumIe***: 0.2	***sumRe***: 0.6133

⋮ ⋮

{g}		
tid	***ie***	***re***
4	0.0625	0
5	0.025	0
inv: 80	***sumIe***: 0.0875	***sumRe***: 0

Fig. 3. Initial efficiency-lists of 1-itemsets.

For our running example, considering an itemset $\{a\}$, its internal efficiency in transaction T_1 is $ie(\{a\},T_1) = 5/44 \approx 0.11$, and its remaining efficiency in transaction T_1 is $re(\{a\},T_1) = 26/44 \approx 0.59$. The sum of internal efficiency is $sumIe(\{a\}) = ie(\{a\},T_1) + ie(\{a\},T_3) + ie(\{a\},T_4) = (5+5+10)/44 \approx 0.45$, while the sum of remaining efficiency is $sumRe(\{a\}) = re(\{a\},T_1) + re(\{a\},T_3) + re(\{a\},T_4) = (26+4+17)/44 \approx 1.07$.

Definition 11 (*Efficiency-List*)**.** An efficiency-list comprises several parts: $\langle E, tid, ie, re, inv, sumIe, sumRe \rangle$, where E is an itemset; *tid* is the transaction *tid* that contains the itemset; *ie* is the internal efficiency of the itemset in transaction *tid*; *re* is the remaining efficiency of this itemset in transaction *tid*; *inv* is the investment that the itemset consumes; *sumIe* is the sum of internal efficiency of this itemset; and *sumRe* is the sum of the remaining efficiency of the itemset. For example, the efficiency-list of all 1-itemsets in D is shown in Fig. 3 (we have kept 4 decimal places).

5.2. Upper bounds and pruning strategies

To reduce the visiting space of a lexicographical tree, we propose other substitutes that can be used to filter redundant nodes. The first substitute is *SEUB*.

Definition 12. Let E be an itemset. We define the efficiency upper bound $SEUB(E)$ as the sum of the internal and remaining efficiency of E, namely, $SEUB(E) = sumIe(E) + sumRe(E)$.

item	b	c	d	e	f	g
a	0.2818	0.563	0.4301	0.4028	0.2153	0.1875
b		0.4752	0.4435	0.4036	0.2831	0.1096
c			0.4839	0.5371	0.2686	0.2774
d				0.3423	0.2081	
e					0.235	0.2389
f						0.0889

Fig. 4. EECS structure.

Theorem 3. *For any itemset X and its extension X'. If $SEUB(X) < \gamma$, then $e(X') < \gamma$.*

Proof. For any transaction T that satisfies $X' \subseteq T$, we have $X \subset X' \subseteq T$, then $X' \backslash X \subseteq T \backslash X$.

Hence,

$$\begin{aligned} ie(X',T) &= \frac{u(X,T)+u(X'\backslash X,T)}{inv(X)+inv(X'\backslash X)} \leqslant \frac{u(X,T)+u(X'\backslash X,T)}{inv(X)} \\ &\leqslant \frac{u(X,T)+u(T\backslash X,T)}{inv(X)} = ie(X,T) + re(X,T). \end{aligned}$$

Since $X \subset X'$, we obtain $tidSet(X') \subset tidSet(X)$. Thus,

$$\begin{aligned} e(X') &= ie(X') = \textstyle\sum_{T\in tidSet(X')} ie(X',T) \\ &\leqslant \textstyle\sum_{T\in tidSet(X')} [ie(X,T) + re(X,T)] \\ &\leqslant \textstyle\sum_{T\in tidSet(X)} [ie(X,T) + re(X,T)] \\ &= SEUB(X). \end{aligned}$$

Pruning strategy 1. *For any itemset X, if $SEUB(X) < \gamma$, then there is no need to explore any extensions X' of X.*

Pruning strategy 2. *Let X be a certain itemset represented by a node N in the lexicographic tree, where X' is represented by a child node of N. If $EUB(X) < \gamma$, we can stop HEPMiner from exploring node N. The reason is that the itemset X' is an extension of itemset X. Hence, $e(X') \leqslant EUB(X') \leqslant EUB(X) < \gamma$. This strategy is used for pruning the unpromising 1-itemsets by EUB.*

The main resource consumption of HEPMiner is the construction of efficiency-lists. During this procedure, the construction of some efficiency-lists is unnecessary and can be eliminated in advance. The following pruning strategy can help to reduce useless construction beforehand. In addition, it depends on a new structure called the estimated efficiency co-occurrence structure (EECS). EECS is a structure to store the *EUB* of every 2-itemset in a transaction database. If we obtain one 2-itemset in the process, this structure can help to find the *EUB* of the 2-itemset quickly. For example, the EECS of the transaction database (Table 2) to store *EUB* is shown in Fig. 4. Since the *EUB* of $\{d, g\}$ is 0, there is no need to store them.

When executing the list construction process, the efficiency-list of extensions is generated. According to the downward closure property of *EUB*, if the *EUB* of the last two items of the original itemset is no more than γ, the *EUB* of extensions of the two items is also no more than γ. Hence, there is no need to perform a list construction process. For example, for list construction of itemsets $\{a,b\}$ and $\{a,c\}$, we want to generate an efficiency-list of itemset $\{a,b,c\}$. They have the same k-1 items ($k = 2$) and are only different at the last item (b and c). If the *EUB* of $\{b,c\}$ is less than γ, which means $e(\{a,b,c\}) \leqslant EUB(\{a,b,c\}) \leqslant EUB(\{b,c\}) < \gamma$, there is no need to generate an efficiency-list for itemset $\{a,b,c\}$. This is summarized in the following pruning strategy 3.

$\{a, c\}$		
tid	***ie***	***re***
1	0.0588	0.1176
3	0.0588	0.0168
4	0.1345	0.0924
inv: 119	***sumIe***: 0.2521	***sumRe***: 0.2269

Fig. 5. Efficiency-list of $\{a, c\}$.

Pruning strategy 3. *Suppose P_x and P_y are two itemsets with the same prefix itemset P, x and y are the last items of P_x and P_y respectively. If there is no tuple (x, y, α) in EECS such that $\alpha \geqslant \gamma$, then there is no need to explore P_{xy} and all its supersets.*

To further filter redundant nodes, we propose another upper bound, *SEUBW*.

Definition 13. Let E be an itemset and $E' = E_x$ be an extension itemset of E by adding one item x. We define the upper bound for width pruning *SEUBW* to be the accumulation of the sum of internal and remaining efficiency of E in transactions that contain E_x, i.e., $SEUBW(E_x) = \sum_{E_x \subset T_p}(ie(E, T_p) + re(E, T_p))$.

Theorem 4. *For any itemset E and its extension itemset E_x by adding one item, if $SEUBW(E_x) < \gamma$, then $e(E_x) < \gamma$.*

Proof. E_x is the extension itemset of E obtained by adding one item; then, we obtain $tidSet(E_x) \subset tidSet(E)$. For any transaction T that satisfies $E_x \subseteq T$, we have $E \subset E_x \subseteq T$. Then, we have

$$\begin{aligned} ie(E_x, T) &= \frac{u(E_x,T)}{inv(E_x)} \leqslant \frac{u(E_x,T)+u(T\setminus E_x,T)}{inv(E_x)} \\ &= \frac{u(E,T)+u(E_x\setminus E,T)+u(T\setminus E_x,T)}{inv(E_x)} \\ &\leqslant \frac{u(E,T)+u(T\setminus E,T)}{inv(E_x)} \leqslant \frac{u(E,T)+u(T\setminus E,T)}{inv(E)} \\ &= ie(E, T) + re(E, T). \end{aligned}$$

Hence, we obtain

$$\begin{aligned} e(E_x) &= ie(E_x) = \sum_{T \in tidSet(E_x)} ie(E_x, T) \\ &\leqslant \sum_{T \in tidSet(E_x)} [ie(E, T) + re(E, T)] \\ &= SEUBW(E_x). \end{aligned}$$

Pruning strategy 4. *Given two itemsets P_x and P_y with the same prefix itemset P, if they satisfy the following condition:*

$$\begin{aligned} SEUBW(P_{xy}) &= sumIe(P_x) + sumRe(P_x) \\ &\quad - \sum_{P_x \subset T_i \wedge P_y \subseteq T_i} [ie(P_x, T_i) + re(P_x, T_i)] \\ &< \gamma, \end{aligned}$$

then there is no need to explore P_{xy} or any extensions thereof.

5.3. Algorithms

In this section, we mainly illustrate the details of the HEPMiner algorithm. HEPMiner is a one-phase method that generates HEIs when visiting the nodes of a lexicographic tree. In addition, it only needs to scan the database two times. Algorithm 4 is the main process of HEPMiner, and it performs for some preparatory work before calling the Search procedure. Algorithm 5 consists of the core idea of HEPMiner. Algorithm 6 describes the process of list combination, which is invoked by Algorithm 5. As shown in Algorithm 4, it scans the database twice for initialization. The first scan calculates *EUB* for every item in the database and uses the *EUB* to filter out unpromising 1-itemsets (pruning strategy 1). The second scan creates an efficiency-list of 1-itemsets and builds the EECS structure. Then, it calls Algorithm 5.

Algorithm 4: HEPMiner

Input: D, a transaction database; PT, a profit table; IT, an investment table; γ, a minimum efficiency threshold.
Output: *HEIs*, the complete set of HEIs.

```
1  first scan D to:
2    (a) calculate investment inv of each item in D;
3    (b) calculate EUB of each 1-itemset in D;
4    (c) put 1-itemsets which satisfy EUB(X) ⩾ γ in C
     (pruning strategy 2);
5    (d) sort 1-itemset in C lexicographically;
6  second scan D to:
7    (a) filter items i satisfying i ∉ C in each transaction ;
8    (b) sort remaining items in each transaction
     lexicographically;
9    (c) create efficiency-list ELs of each 1-itemset in C and build
     the EECS;
10 call Search(∅, ∅, ELs, eecs, γ);
```

Algorithm 5 needs the prefix itemset P, the efficiency-list of P, efficiency-lists of extensions of P, the EECS structure, and minimum efficiency threshold γ as input. It uses *sumIe* to determine HEIs and *SEUB* to decide if there is a need to explore the extensions (pruning strategy 1). If there is a need to explore the extensions, it must also judge the necessity of performing list combination by EECS (pruning strategy 3). It then produces efficiency-lists of extensions of P and continues the next recursion if both conditions are met.

Algorithm 5: Search

Input: P, a prefix itemset; PL, efficiency-list of P; *ELs*, efficiency-lists of extensions; *eecs*, EECS structure; γ, a minimum efficiency threshold.
Output: *HEIs*, the complete set of HEIs.

```
1  for PL_x ∈ ELs do
2    if sumIe(PL_x) ⩾ γ then
3      output PL_x.itemset;
4    end
5    if sumIe(PL_x) + sumRe(PL_x) ⩾ γ (pruning strategy 1) then
6      exELs = null;
7      for PL_y ∈ ELs ∧ PL_y.itemset ≻ PL_x.itemset do
8        if ∃(x, y, α) ∈ eecs for storing EUB such that α ⩾ γ
           (pruning strategy 3) then
9          exELs = exULs + Construct(PL, PL_x, PL_y);
10       end
11     end
12     call Search(PL_x.itemset, PL_x, exELs, eecs, γ);
13   end
14 end
```

Algorithm 6 produces the efficiency-list of the extension itemsets. In particular, it adopts the pruning strategy 4 in pseudocode. For example, the combination of efficiency-lists of the itemset $\{a, c\}$ is shown in Fig. 5. It comes from the efficiency-list of itemset $\{a\}$ and $\{c\}$ (in Fig. 3).

Algorithm 6: Construct

Input: P, efficiency-list of P; P_x, efficiency-list for extension of P with an item x; P_y, efficiency-list for extension of P with an item y.
Output: P_{xy}, efficiency-list for extension of P with $\{x, y\}$.

```
1  P_xy.inv = P_x.inv + P_y.inv;
2  if P ≠ null then
3      P_xy.inv = P_xy.inv − P.inv;
4  end
5  SEUBW = sumIe(P_x) + sumRe(P_x);
6  for each element E_x ∈ P_x do
7      if ∃ element E_y ∈ P_y satisfying E_x.tid = E_y.tid then
8          if P ≠ null ∧ ∃ element E_p ∈ P satisfy E_p.tid = E_x.tid
             then
9              E_xy.ie = (E_x.ie × P_x.inv + E_y.ie × P_y.inv − E_p.ie ×
                 P.inv) / P_xy.inv;
10             E_xy.re = (E_y.re × P_y.inv) / P_xy.inv;
11             E_xy = ⟨E_x.tid, E_xy.ie, E_xy.re⟩;
12         else
13             E_xy.ie = (E_x.ie × P_x.inv + E_y.ie × P_y.inv) / P_xy.inv;
14             E_xy.re = (E_y.re × P_y.inv) / P_xy.inv;
15             E_xy = ⟨E_x.tid, E_xy.ie, E_xy.re⟩;
16         end
17         append E_xy to P_xy;
18     else
19         SEUBW = SEUBW −(E_x.ie + E_x.re);
20         if SEUBW < γ (pruning strategy 4) then
21             return null
22         end
23     end
24 end
25 return P_xy
```

6. Experiment evaluation

In this section, several experiments were conducted on different datasets to demonstrate the performance of our algorithms. Since this is the first study devoted to discovering novel efficiency patterns, we chose the HUIM method as our comparison method to observe the final result patterns in the case study subsection. In addition, we compare our two proposed algorithms and their distinct pruning strategies. In this section, HEPMiner-1 refers to the HEPMiner algorithm using pruning strategies 1 to 2, HEPMiner-2 refers to the HEPMiner algorithm using pruning strategies 1–3, and HEPMiner-3 refers to the HEPMiner algorithm using pruning strategies 1–4. We examine the properties of the two algorithms and their corresponding pruning strategies by observing indicators of runtime, memory usage, candidate space, joint counts, and pattern numbers. In addition, we also apply our algorithms to several datasets of different sizes to test their scalability.

6.1. Datasets and experimental settings

To illustrate the performance of our methods, we select seven datasets with different properties. In particular, the last dataset is used to examine the scalability of methods since it is large in size. The selected datasets are derived from real datasets by adding synthetic internal utility and investment values. The synthetic internal utility values are downloaded from the SPMF platform. For investment values, since many phenomena in the world follow the laws of Gaussian distribution, we utilize the Gaussian distribution $N(10000, 100^2)$ to generate them. The large variance adds diversity to the data. If the obtained investment value is negative, we use the Gaussian distribution with $N(100, 5^2)$ to generate a small investment value. Table 5 illustrates the features of those datasets. $|D|$ represents the total transaction number in D, $|I|$ represents the distinctive item amount in the dataset, *AvgLen* represents the average transaction length for the dataset, and *Density* represents the density of the dataset, which is computed by *AvgLen* divided by $|I|$.

Table 5
Characteristics of the datasets.

Dataset	$\|D\|$	|I|	AvgLen	Density
foodmart	21556	1559	4.42	0.28%
retail	88162	16470	10.30	0.06%
mushroom	8416	119	23	19.33%
chess	3196	75	37	49.33%
accidents	340183	468	33.8	7.22%
BMS	77512	3340	4.62	0.14%
chainstore	1112949	46086	7.23	0.02%

Both of our algorithms are implemented in Java on the framework JDK 8 without other dependencies. In addition, we use Apache Maven 3.6.3 to build and manage our project. The codes can be found in our GitHub repository.[1] The corresponding experiments are conducted on a personal computer. The basic configurations of the laptop are as follows: Intel Core i5-1035G1 CPU1.00 GHz, 16 GB of RAM, and 64-bit Microsoft Windows 10 OS. We measure runtime consumption, memory usage, joint count, candidates, and pattern numbers under different minimum efficiency threshold values γ to illustrate the properties of our algorithms and corresponding pruning strategies. It is worth mentioning that the joint count mainly evaluates the performance of the pruning strategies of HEPMiner.

6.2. Runtime analysis

To assess the performance of our algorithms and pruning strategies, we select a runtime indicator under a different minimum efficiency threshold value γ for observation. Figs. 6 and 7 illustrate the final observation results on the different datasets.

We begin by examining the runtime of HEPM and HEPMiner at various minimum efficiency threshold values (γ). For each dataset with different properties, HEPMiner always consumes less time than HEPM, and the time difference between HEPMiner and HEPM is significant. This disparity is approximately dozens of seconds for foodmart, hundreds of seconds for mushroom, chess, accidents, and BMS, and thousands of seconds for retail. This observation indicates HEPMiner always outperforms HEPM since it consumes less time under different efficiency threshold values and in different datasets. In each dataset, as γ increases, the time disparity tends to decrease. This is due to a relatively strict large threshold to confine the resulting pattern number, and a few patterns cannot consume much time. We believe that if the threshold were large enough, the running times would tend to be similar. In each dataset with different properties, both HEPM and HEPMiner consume less runtime. This is rational since a large threshold is a relatively strict constraint, which filters certain unnecessary computations of the methods. The decreasing trend of HEPM is more evident than that of HEPMiner since HEPMiner always consumes less runtime, especially for small efficiency threshold values. We can see a clear decreasing tendency on the retail and accidents datasets from Fig. 7.

For distinct pruning strategies of the HEPMiner algorithm, there is no noticeable time difference for the retail, mushroom, chess, and BMS datasets under different efficiency thresholds. However, after enlarging

[1] https://github.com/homexiaojie888/HEPM

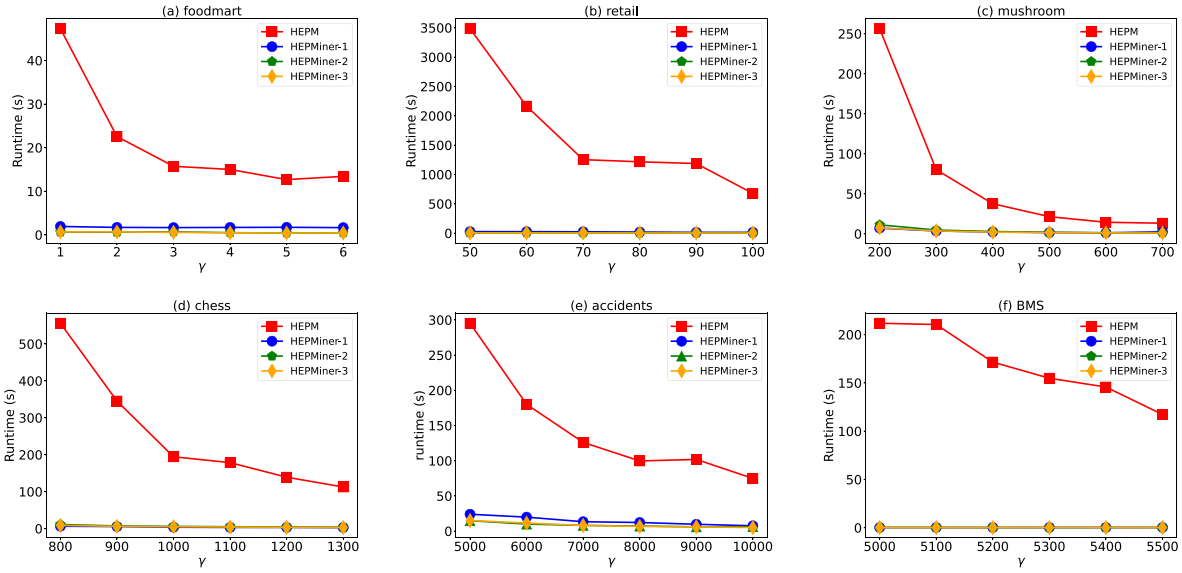


Fig. 6. Runtime when varying γ. (a) foodmart, (b) retail, (c) mushroom, (d) chess, (e) accidents, and (f) BMS.

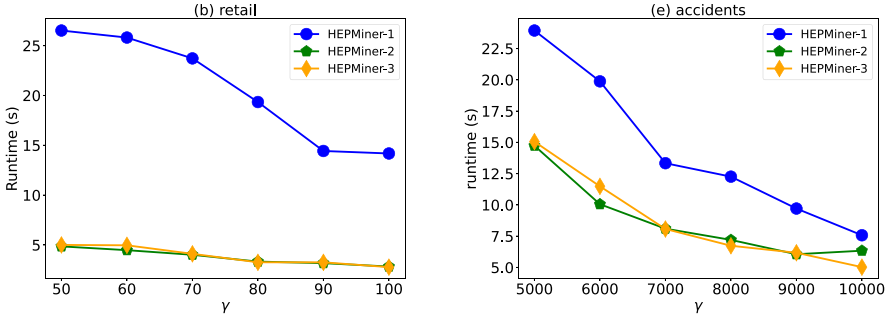


Fig. 7. Runtime when varying γ. (b) retail, and (e) accidents.

the foodmart and accidents datasets in Fig. 7, we find that HEPMiner-2 and HEPMiner-3 are tens of seconds faster than HEPMiner-1. This implies that proposed pruning strategies 3 and 4 play an important role in reducing time consumption. There is no remarkable time difference between HEPMiner-2 and HEPMiner-3, which means that pruning strategy 4 contributes little in terms of saving time. We also find that HEPMiner-1, HEPMiner-2, and HEPMiner-3 consume less time as the efficiency threshold increases.

6.3. Memory evaluation

To evaluate the performance of the algorithms and pruning strategies, we examine a memory usage indicator at various minimum efficiency threshold values (γ). Figs. 8 and 9 demonstrate the related results in different datasets.

First, we focus on the memory consumption of HEPM and HEPMiner under different minimum efficiency thresholds γ. Except for the foodmart dataset, HEPM always uses more memory than HEPMiner under different efficiency threshold values. This memory difference is more evident for dense datasets such as the mushroom, chess, and BMS datasets. For example, the gap in memory consumption is thousands of megabytes at most on the mushroom, chess, and BMS datasets. On sparse datasets, the gap is not distinct. For example, the difference between HEPM and HEPMiner is hundreds of megabytes on the retail and BMS datasets. On the foodmart dataset, HEPM consumes distinctly less memory than HEPMiner when γ is 2 or 3. This result illustrates that HEPMiner always works better in dense datasets. For sparse datasets, the results are not always certain. On dense datasets with different properties, HEPM consumes less and less memory as the efficiency threshold increases. This is rational since a large threshold is a relatively strict constraint, which filters certain unnecessary computations of the methods. Note that the decreasing trend of HEPMiner is not evident since HEPMiner always consumes less memory even with small efficiency threshold values. The memory consumption of HEPM and HEPMiner does not show obvious ascent or descent regularity with increasing efficiency threshold on sparse datasets with different properties. HEPM fluctuates slightly, but HEPMiner remains relatively stable as the efficiency threshold increases.

For different pruning strategies of HEPMiner, there is no significant discrepancy in memory usage under different efficiency threshold values except for the accidents dataset. For example, the difference on the foodmart and BMS datasets remains in the tens of megabits. In contrast, the difference on the retail, mushroom, and chess datasets remains in the hundreds of megabits. In addition, different pruning strategies of HEPMiner do not show apparent regularity as efficiency thresholds increase, except for the accidents dataset. This is normal since HEPMiner does not use more than 1000 megabits of memory, and the Java garbage collection mechanism may cause fluctuations under the condition of less memory loss. For the accidents dataset, HEPMiner-1 always uses less memory than HEPMiner-2 and HEPMiner-3. This is due to the large size and density of accidents leading to EECS consuming more memory by HEPMiner-2 and HEPMiner-3. In addition, in most cases, HEPMiner-2 and HEPMiner-3 usually share the same memory usage, which means that pruning strategy 4 usually consumes little memory.

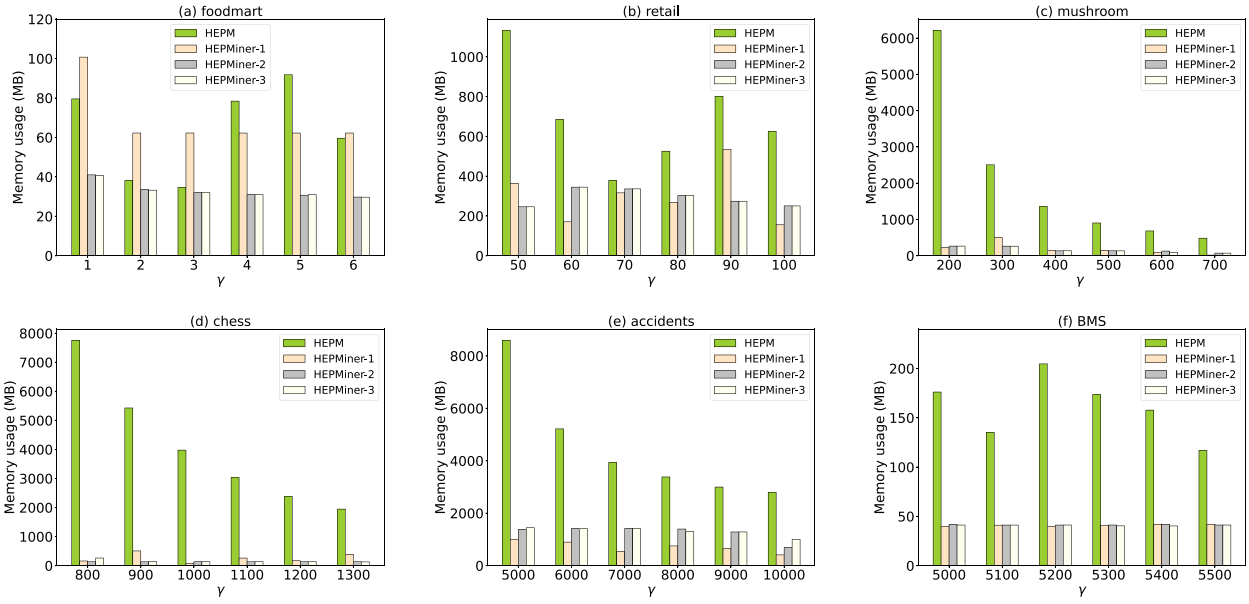


Fig. 8. Memory when varying γ. (a) foodmart, (b) retail, (c) mushroom, (d) chess, (e) accidents, and (f) BMS.

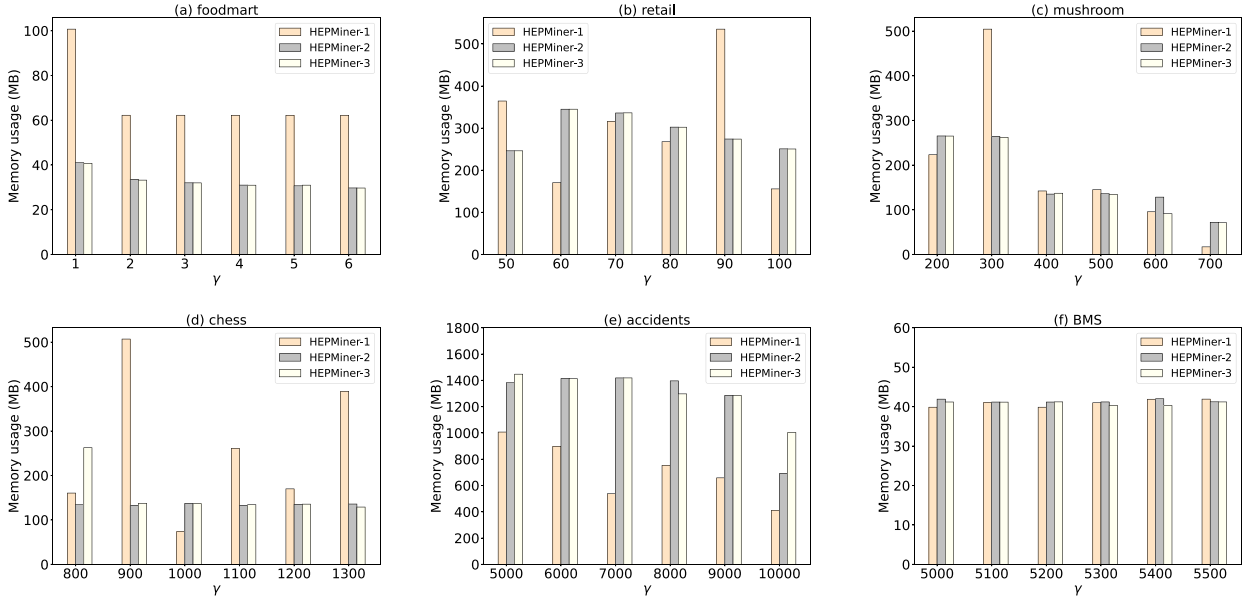


Fig. 9. Memory when varying γ. (a) foodmart, (b) retail, (c) mushroom, (d) chess, (e) accidents, and (f) BMS.

6.4. Candidate and patterns analysis

To evaluate the performance of our algorithms and pruning strategies, we select the number of candidates and patterns as indicators under different minimum efficiency threshold values γ for testing. Figs. 10 and 11 demonstrate the useful results in different datasets. Fig. 10 describes the candidate number and pattern number of the compared algorithms, while Fig. 11 magnifies the unclear details of some datasets.

For candidate numbers, we first analyze the situation of HEPM and HEPMiner and then reflect on the function of pruning strategies among HEPMiner. Except for the chess and BMS datasets, the HEPMiner-1 algorithm always visits more candidate nodes than the others, while HEPM has the highest candidate number in the BMS datasets. For the chess dataset, the candidate number visited by HEPMiner-1 surpasses HEPM when the efficiency threshold value γ equals 1100. This is rational since HEPM reduces the unpromising node in the current level, while HEPMiner filters the unpromising child nodes in the next level. In most cases, pruning at the current level is more efficient than pruning at the next level. However, this depends on other factors, such as dataset features and the efficiency threshold value. Concerning the different pruning strategies of HEPMiner, HEPMiner-1 always visits the most candidates on all datasets, HEPMiner-2 always ranks second on all datasets, and HEPMiner-3 ranks third except for the retail and BMS datasets. This means that pruning strategy 3 can significantly reduce the candidate space, and pruning strategy 4 can reduce it even further.

For pattern numbers, we find that all compared algorithms and pruning strategies derive the same number of patterns under different minimum efficiency threshold values γ, which shows the completeness of our obtained patterns. In addition, we compare the patterns derived from HEPM and HEPMiner-1 to HEPMiner-3, which are the same, showing the correctness of the algorithms and pruning strategies.

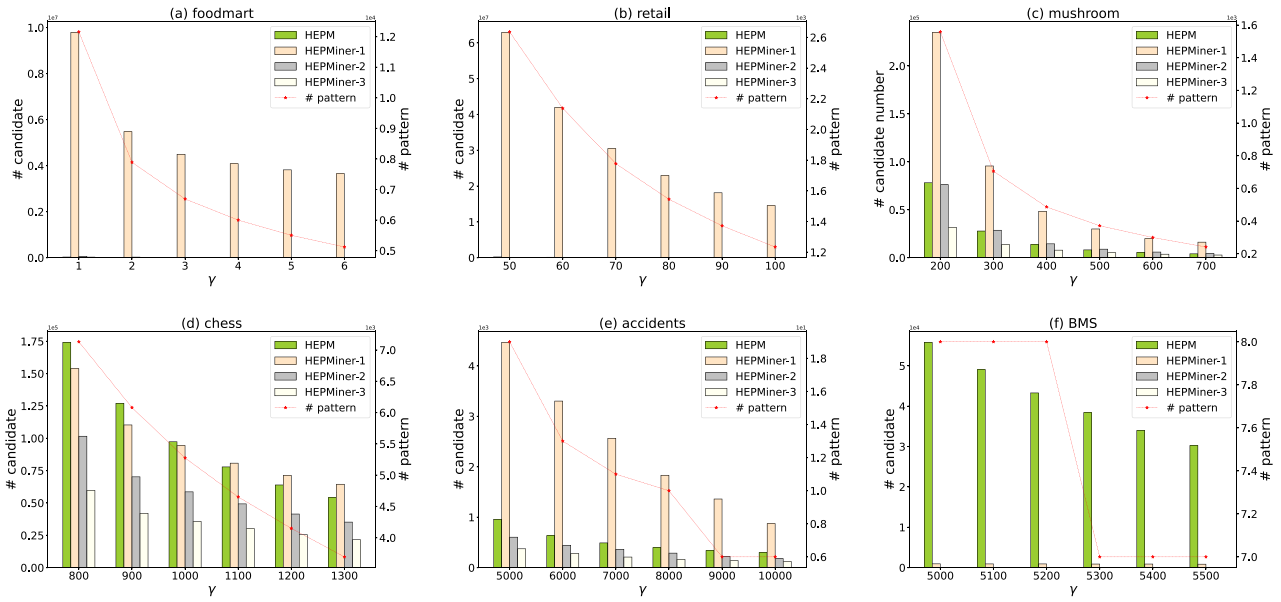


Fig. 10. Candidate and pattern number when varying γ. (a) foodmart, (b) retail, (c) mushroom, (d) chess, (e) accidents, and (f) BMS.

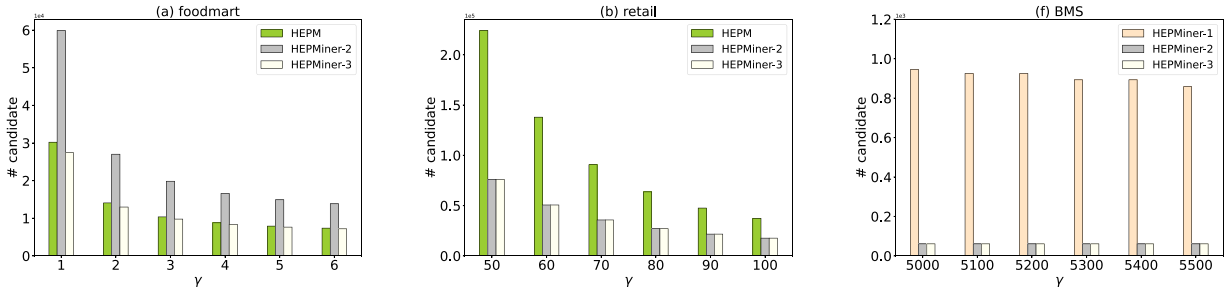


Fig. 11. Candidate number when varying γ. (a) foodmart, (b) retail, and (f) BMS.

6.5. Joint number analysis

To demonstrate the performance of the HEPMiner pruning strategies, we take the number of joint counts as an indicator under different minimum efficiency threshold values (γ). The joint counts represent the number of the construction processes (Algorithm 6) executed. Since some pruning strategies may reduce the number of construction processes to reduce time consumption, we use only a variable in the realization code of Algorithm 6 to count the number of times that it is called. Note that strategies of joint count only exist in HEPMiner. Since they depend on the efficiency-list structure, Figs. 12 and 13 demonstrate the final results in different datasets. Fig. 12 describes the joint count of pruning strategies 3 and 4, while Fig. 13 magnifies the unclear details of some datasets. HEPMiner-2 significantly reduces the joint number compared to HEPMiner-1 on all datasets under different efficiency threshold values. On the foodmart and retail datasets, the reduced number attends to 10^7; on the mushroom and chess datasets, it is 10^5; and on the accidents and BMS datasets, it is 10^3. This means that pruning strategy 3 is effective and can significantly reduce the candidate space. We also conclude that HEPMiner-3 can effectively eliminate joint numbers compared with HEPMiner-2 except for the retail and BMS datasets. The reduced number on the foodmart, mushroom, and chess datasets is 10^4; on accidents, it is 10^2. This illustrates that pruning strategy 4 is effective in certain cases and can further reduce the search space.

6.6. Pattern comparison

To further test the effectiveness of HEPM and HEPMiner, we compare the content of patterns to illustrate the difference between HUIM and HEPM. Many algorithms [10,29] aim to find HUIs in various datasets, and one can obtain the final result by using one of these algorithms. To contrast the differences easily, we first chose a small dataset in Table 2 as a test dataset and then applied our methods and HUIM methods to it. The generated top 10 HUIs and HEIs are shown in Table 1. We observe that the obtained HUIs and HEIs are distinct itemsets. For instance, $\{b, e\}$ is both HEIs and HUIs, which means that $\{b, e\}$ has both high efficiency and utility and deserves to continue operating. $\{a\}$ is HEIs but not HUIs, which means that $\{a\}$ has high efficiency and low utility and may continue operating since there is a lower investment. $\{b, c, e\}$ is HUIs but not HEIs, which means that $\{b, c, e\}$ has low efficiency and high utility and may not be able to continue operating since it needs high investment. HEPM is devoted to finding high-efficiency itemsets with low or high utility.

When the datasets are the real big datasets as shown in Table 5, the obtained HEIs may experience a large shuffle compared to the HUIs. We next apply our methods and HUIM methods [10,29] to these real datasets. To save space, we also keep only the top 10 patterns for both of them; the generated top 10 HUIs and HEIs are shown in Fig. 14. Note that datasets have been preprocessed to represent items by numbers, not by their original names. On the foodmart dataset, the top 1 HUI is the itemset {1428}, but we do not find {1428} in the top 10 HEIs since the large investment makes its ranking decrease in efficiency. We also find in the chess and mushroom datasets that the top 10 HUIs have long

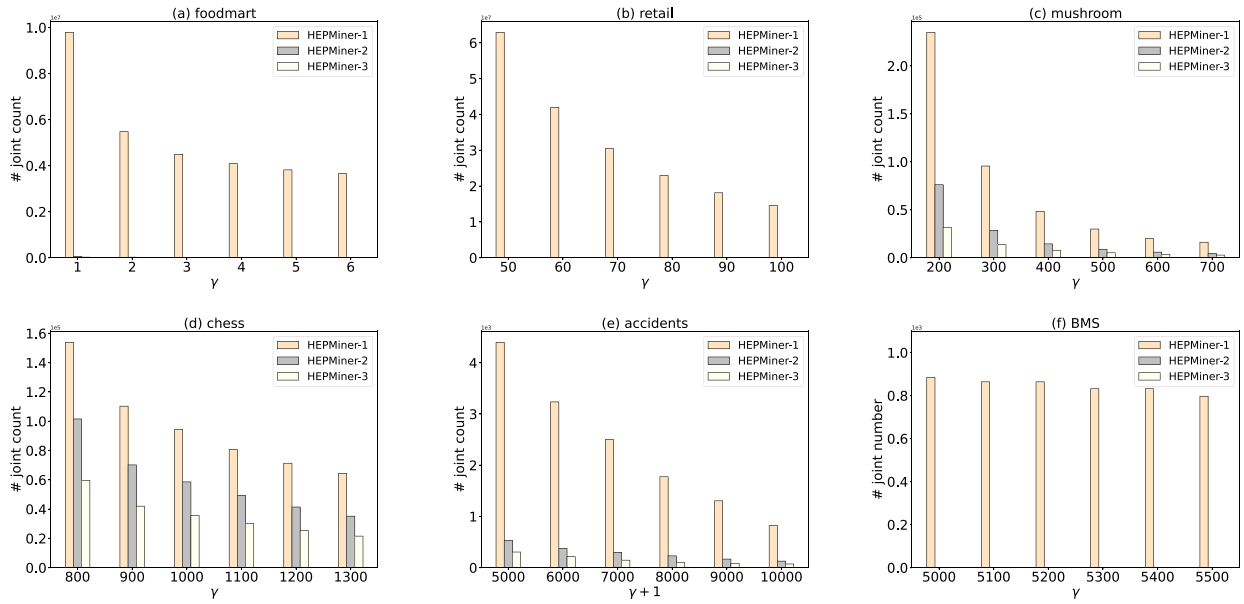


Fig. 12. Joint number when varying γ. (a) foodmart, (b) retail, (c) mushroom, (d) chess, (e) accidents, and (f) BMS.

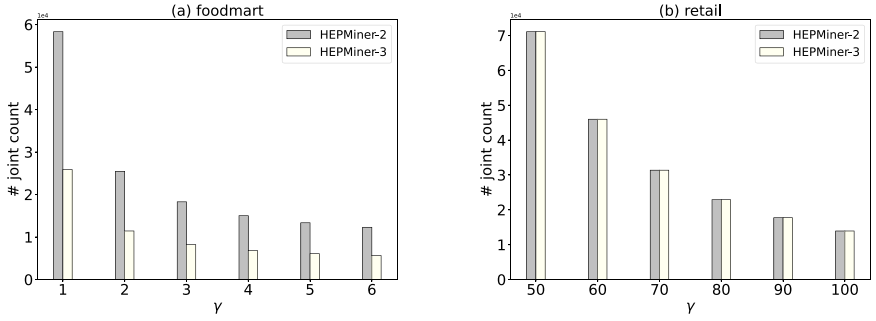


Fig. 13. Joint number when varying γ. (a) foodmart, and (b) retail.

lengths. Regarding the top 10 HEIs, the length is inclined to be short. This is because long itemsets are more likely to have more significant investment.

6.7. Scalability

To assess the scalability of HEPM and HEPMiner, we select runtime and memory as indicators under different database sizes for testing. Fig. 15 demonstrates the final results. The first and second figures in Fig. 15 describe the runtime of our algorithms in different database sizes (data amount expands five times each time). The third demonstrates the memory usage of our algorithms in different database sizes (the data amount expands five times).

To check the scalability of our method to different database sizes, we utilize the last chainstore dataset to generate new datasets with different transaction records by randomly adding or reducing transactions coming from it. The runtime of our algorithms in different database sizes is shown in Fig. 15. We conclude that as transaction amounts increase, both HEPM and HEPMiner-4 increase in time consumption. All of them display nearly linear relationships with the increase in the number of transactions. Fig. 15 shows the memory usage of our algorithms in different database sizes. We conclude that as the number of transactions increases, both HEPM and HEPMiner-4 do not always experience increased memory consumption, which is partly derived from the garbage collection mechanism of the Java Virtual Machine.

7. Conclusions

This paper has made contributions in both theoretical and practical aspects. Theoretically, we define a new problem model to find the most investment-efficient patterns by simultaneously considering utility and investment. We proposed two algorithms and corresponding techniques (storage, pruning strategies, and so on) to address the problem. For practical contributions, our problem model and corresponding algorithms can be used in marketing analysis to find high-efficiency patterns, and these patterns can help leaders make decisions. Relevant personnel can use the mined patterns to analyze their properties and then take strategies to increase profit-to-invest.

We originally formulated the problem model of high-efficiency pattern mining and proposed two algorithms (HEPM and HEPMiner) to solve this problem. The HEPM algorithm is a two-phase algorithm, while HEPMiner is a one-phase algorithm. For HEPM, we create an upper bound *EUB* and design corresponding pruning strategies to reduce the search space. Regarding HEPMiner, we developed a novel list structure called the efficiency-list to make HEPMiner one-phase. In addition, new upper bounds *SEUB* and *SEUBW* and four pruning strategies are designed to increase the performance of the HEPMiner algorithm. The experimental results illustrate the effectiveness and properties of our two algorithms. In future work, there are two main aspects to explore. On the one hand, we plan to design more efficient methods to find HEIs,

foodmart		retail		mushroom	
HUIs: utility ↓	HEIs: efficiency ↓	HUIs: utility ↓	HEIs: efficiency ↓	HUIs: utility ↓	HEIs: efficiency ↓
{1428}: 187137	{86}: 7920.66	{48}: 28149600	{38}: 76832.0	{48}: 28149600	{93}: 23995.73
{11}: 163982	{572}: 4347.0	{39, 48}: 23388528	{16011}: 12087.10	{39, 48}: 23388528	{9}: 15896.09
{975}: 159127	{1290}: 2140.0	{41, 48}: 10039311	{475}: 10632.0	{41, 48}: 10039311	{76}: 15191.52
{712}: 158631	{464}: 2126.02	{39, 41, 48}: 9210739	{209}: 8414.56	{39, 41, 48}: 9210739	{10}: 14946.96
{390}: 157070	{1049}: 1931.67	{38, 48}: 8015164	{36, 38}: 7574.67	{38, 48}: 8015164	{76, 93}: 12134.99
{1380}: 154833	{952}: 1916.54	{38, 39, 48}: 6985672	{255}: 7420.18	{38, 39, 48}: 6985672	{9, 76}: 8758.6
{1448}: 153180	{1525}: 1665.82	{39}: 6838110	{2983}: 7116.16	{39}: 6838110	{69}: 8560.77
{1121}: 149171	{1142}: 1640.61	{41}: 6659406	{6004}: 7037.5	{41}: 6659406	{10, 93}: 7924.98
{362}: 148212	{1342}: 1618.60	{39, 41}: 6646950	{10}: 6290.90	{39, 41}: 6646950	{9, 93}: 7447.33
{116}: 147246	{1024}: 1600.27	{32, 48}: 6020704	{19}: 5433.17	{32, 48}: 6020704	{3, 93}: 6715.45

chess		accidents		BMS	
HUIs: utility ↓	HEIs: efficiency ↓	HUIs: utility ↓	HEIs: efficiency ↓	HUIs: utility ↓	HEIs: efficiency ↓
A_1: 10305887	{48}: 17141.03	B_1: 29450265	{46}: 48130.47	{168}: 9503520	{112}: 67254.5
A_2: 10287149	{40, 48}: 15449.51	B_2: 29187899	{7}: 29551.42	{112}: 4371543	{299}: 27557.8
A_3: 10148707	{48, 58}: 14813.3	B_3: 28605953	{18, 46}: 22981.18	{119}: 3157056	{324}: 10721.13
A_4: 10100515	{40}: 14461.28	B_4: 28528939	{7, 18}: 14768.10	{306}: 2848101	{386}: 10131.9
A_5: 10087096	{40, 48, 58}: 14356.11	B_5: 28478922	{44}: 14424.0	{317}: 2682790	{149}: 9225.62
A_6: 10078951	{40, 58}: 13622.05	B_6: 28477711	{18}: 13055.75	{72}: 2350460	{318}: 8428.33
A_7: 10062405	{58}: 12934.17	B_7: 28196647	{99}: 8930.58	{315}: 2319526	{99}: 7364.50
A_8: 10055911	{40, 48, 62}: 12157.76	B_8: 28122366	{44, 46}: 8670.09	C_1: 2004812	{101}: 5217.07
A_9: 10038909	{40, 48, 58, 62}: 12022.71	B_9: 28075276	{7, 46}: 8050.41	C_2: 2004594	{67}: 4397.73
A_{10}: 10038276	{11}: 11922.68	B_{10}: 27649347	{18, 44}: 8031.06	C_3: 2004536	{198}: 4089.35

Fig. 14. Top 10 HUIs and HEIs in different datasets. Here, items are represented by numbers, not by their original names. A_i, B_i and C_i are itemsets obtained. For example, $A_1 = \{5, 9, 29, 34, 36, 40, 48, 52, 58, 60, 66\}$, $A_2 = \{5, 9, 29, 34, 36, 40, 48, 52, 58, 60, 62, 66\}, \ldots$, $B_1 = \{12, 16, 17, 18, 21, 28, 31, 4\}$, $B_2 = \{12, 16, 17, 18, 28, 31, 4\}, \ldots$. C_1, C_2, C_3 are long itemsets with many items, for example, $C_1 = \{249, 224, 174, \ldots\}$.

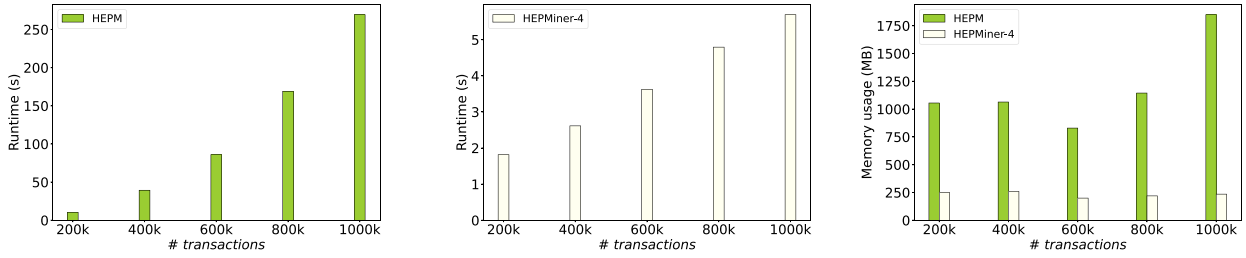


Fig. 15. Runtime (first and second) and memory (third) when varying transaction number.

including creating improved structures or inventing more effective pruning strategies. On the other hand, we want to find efficient patterns in more complicated data types, such as sequences, episodes, graphs, and dynamic data, or based on targeted objects.

CRediT authorship contribution statement

Xiaojie Zhang: Writing – original draft, Software, Methodology, Conceptualization. **Guoting Chen:** Writing – review & editing, Supervision, Funding acquisition. **Linqi Song:** Writing – review & editing, Supervision. **Wensheng Gan:** Writing – review & editing, Supervision. **Yunling Song:** Writing – review & editing, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Datas used in this paper are publicly available on internet.

Acknowledgments

This work is supported in part by the Shenzhen Science and Technology Program (Basic Research Project, No. JCYJ2021032413300

3011), the National Natural Science Foundation of China (Nos. 62002136, 62272196, and 12171491), the Natural Science Foundation of Guangdong Province (No. 2022A1515011861), and the Hong Kong RGC-CityU RMGS 9229104. Linqi Song is a corresponding author.

References

[1] R. Agrawal, R. Srikant, Fast algorithms for mining association rules in large databases, in: Proc. 20th Intern. Conf. Very Large Data Bases, 1994, pp. 487–499.

[2] J. Han, J. Pei, Y. Yin, R. Mao, Mining frequent patterns without candidate generation: A frequent-pattern tree approach, Data Mining Knowl. Discov. 8 (1) (2004) 53–87.

[3] Y. Liu, W. Liao, A.N. Choudhary, A two-phase algorithm for fast discovery of high utility itemsets, in: Proc. 9th Pacific-Asia Conf. Advances in Knowl. Discovery and Data Mining, in: Lecture Notes in Computer Science, vol.3518, 2005, pp. 689–695.

[4] J. Pei, J. Han, B. Mortazavi-Asl, H. Pinto, Q. Chen, U. Dayal, M. Hsu, PrefixSpan: Mining sequential patterns by prefix-projected growth, in: Proc. 17th Intern. Conf. Data Engin., 2001, pp. 215–224.

[5] Y.-H. Hu, T.-W. Yeh, Discovering valuable frequent patterns based on RFM analysis without customer identification information, Knowl.-Based Syst. 6 (2014) 76–88.

[6] M. Zihayat, H. Davoudi, A. An, Mining significant high utility gene regulation sequential patterns, BMC Syst. Biol. 11 (6) (2017) 109.

[7] A. Aggarwal, D. Toshniwal, Spatio-temporal frequent itemset mining on web data, in: Proc. 2018 IEEE Intern. Conf. Data Mining Workshops, 2018, pp. 1160–1165.

[8] J. Huang, J. Huan, A. Tropsha, J. Dang, H. Zhang, M. Xiong, Semantics-driven frequent data pattern mining on electronic health records for effective adverse drug event monitoring, in: Proc. 2013 IEEE Intern. Conf. Bioinformatics and Biomedicine, 2013, pp. 608–611.

[9] K. Divyavarma, M. Remya, G. Deepa, An enhanced bug mining for identifying frequent bug pattern using word tokenizer and FP-Growth, in: Proc. 5th Intern. Conf. Frontiers in Intell. Computing: Theory and Appl., in: Advances in Intelligent Systems and Computing, vol.515, 2016, pp. 525–532.

[10] M. Liu, J. Qu, Mining high utility itemsets without candidate generation, in: Proc. 21st ACM Intern. Conf. on Information and Knowl. Management, 2012, pp. 55–64.

[11] N. Aryabarzan, B. Minaei-Bidgoli, NEclatClosed: A vertical algorithm for mining frequent closed itemsets, Expert Syst. Appl. 174 (2021) 114738.

[12] R.U. Kiran, P. Fournier-Viger, J.M. Luna, J.C.-W. Lin, A. Mondal (Eds.), Periodic Pattern Mining, Theory, Algorithms, and Applications, Springer, 2021.

[13] X. Zhang, Y. Qi, G. Chen, W. Gan, P. Fournier-Viger, Fuzzy-driven periodic frequent pattern mining, Inform. Sci. 618 (2022) 253–269.

[14] W. Gan, J.C. Lin, H. Chao, H. Fujita, P.S. Yu, Correlated utility-based pattern mining, Inform. Sci. 504 (2019) 470–486.

[15] W. Gan, J.C. Lin, P. Fournier-Viger, H. Chao, H. Fujita, Extracting non-redundant correlated purchase behaviors by utility measure, Knowl.-Based Syst. 143 (2018) 30–41.

[16] M. Nouioua, P. Fournier-Viger, J. Qu, J.C. Lin, W. Gan, W. Song, CHUQI-Miner: Mining correlated quantitative high utility itemsets, in: Proc. 2021 Intern. Conf. on Data Mining, IEEE, 2021, pp. 599–606.

[17] J. Syu, J.C. Lin, C. Wu, J. Ho, Stock selection system through suitability index and fuzzy-based quantitative characteristics, IEEE Trans. Fuzzy Syst. 31 (1) (2023) 322–334.

[18] X. Ao, P. Luo, C. Li, F. Zhuang, Q. He, Online frequent episode mining, in: Proc. 31st IEEE Intern. Conf. Data Engin., 2015, pp. 891–902.

[19] J.C. Lin, W. Gan, T. Hong, H. Chen, S. Li, An efficient algorithm to maintain the discovered frequent sequences with record deletion, Intell. Data Anal. 20 (3) (2016) 655–677.

[20] M.J. Zaki, Scalable algorithms for association mining, IEEE Trans. Knowl. Data Eng. 12 (3) (2000) 372–390.

[21] P. Fournier-Viger, Y. Wang, P. Yang, C.W. Lin, R.U. Kiran, TSPIN: Mining top-k stable periodic patterns, Appl. Intell. 52 (6) (2022) 6917–6938.

[22] R.U. Kiran, C. Saideep, K. Zettsu, M. Toyoda, M. Kitsuregawa, P.K. Reddy, Discovering partial periodic spatial patterns in spatiotemporal databases, in: Proc. 2019 IEEE Intern. Conf. Big Data, 2019, pp. 233–238.

[23] T. Le, B. Vo, An N-list-based algorithm for mining frequent closed patterns, Expert Syst. Appl. 42 (19) (2015) 6648–6657.

[24] C. Wu, J. Huang, Y. Lin, C. Chuang, Y. Tseng, Efficient algorithms for deriving complete frequent itemsets from frequent closed itemsets, Appl. Intell. 52 (6) (2022) 7002–7023.

[25] X. Liu, G. Chen, W. Zuo, Effective algorithms to mine skyline frequent-utility itemsets, Eng. Appl. Artif. Intell. 116 (2022) 105355.

[26] Y. Chen, W. Gan, Y. Wu, P.S. Yu, Privacy-preserving federated mining of frequent itemsets, Inform. Sci. 625 (2023) 504–520.

[27] J. Li, W. Gan, Y. Gui, Y. Wu, P.S. Yu, Frequent itemset mining with local differential privacy, in: Proc. 31st ACM Intern. Conf. on Information & Knowledge Management, 2022, pp. 1146–1155.

[28] W. Liang, H. Chen, J. Zhang, D. Zhao, C. Li, An effective scheme for top-k frequent itemset mining under differential privacy conditions, Sci. China Inf. Sci. 63 (5) (2020).

[29] P. Fournier-Viger, C. Wu, S. Zida, V.S. Tseng, FHM: Faster high-utility itemset mining using estimated utility co-occurrence pruning, in: Proc. 21st Intern. Symp. Foundations of Intell. Systems, in: Lecture Notes in Computer Science, vol.8502, 2014, pp. 83–92.

[30] W. Gan, J.C. Lin, P. Fournier-Viger, H. Chao, P.S. Yu, HUOPM: High-utility occupancy pattern mining, IEEE Trans. Cybern. 50 (3) (2020) 1195–1208.

[31] C. Wu, B. Shie, V.S. Tseng, P.S. Yu, Mining top-k high utility itemsets, in: Proc. 18th ACM SIGKDD Intern. Conf. Knowl. Discovery and Data Mining, 2012, pp. 78–86.

[32] J.C. Lin, W. Gan, T. Hong, J. Pan, Incrementally updating high-utility itemsets with transaction insertion, in: Proc. 10th Intern. Conf. Advanced Data Mining and Applications, in: Lecture Notes in Computer Science, vol.8933, 2014, pp. 44–56.

[33] P. Fournier-Viger, S. Zida, FOSHU: Faster on-shelf high utility itemset mining - with or without negative unit profit, in: Proc. 30th Annual ACM Symp. Applied Computing, 2015, pp. 857–864.

[34] H. Kim, T. Ryu, C. Lee, H. Kim, E. Yoon, B. Vo, J.C. Lin, U. Yun, EHMIN: Efficient approach of list based high-utility pattern mining with negative unit profits, Expert Syst. Appl. 209 (2022) 118214.

[35] S. Krishnamoorthy, Efficiently mining high utility itemsets with negative unit profits, Knowl.-Based Syst. 145 (2018) 1–14.

[36] S. Huang, W. Gan, J. Miao, X. Han, P. Fournier-Viger, Targeted mining of top-k high utility itemsets, 2023, arXiv:2303.14510.

[37] P. Zhang, J. Chen, S. Wan, W. Gan, Targeted mining of rare high-utility patterns, in: IEEE Intern. Conf. on Big Data, 2022, pp. 6271–6280.

[38] V.S. Tseng, C. Wu, B. Shie, P.S. Yu, UP-growth: An efficient algorithm for high utility itemset mining, in: Proc. 16th ACM SIGKDD Intern. Conf. Knowl. Discovery and Data Mining, 2010, pp. 253–262.

[39] P. Fournier-Viger, J.C.-W. Lin, Q.-H. Duong, T.-L. Dam, PHM: Mining periodic high-utility itemsets, in: Proc. Industrial Conf. Data Mining, in: Lecture Notes in Computer Science, vol. 9728, 2016, pp. 64–79.

[40] T.Y. Reddy, R.U. Kiran, M. Toyoda, P.K. Reddy, M. Kitsuregawa, Discovering partial periodic high utility itemsets in temporal databases, in: S. Hartmann, J. Küng, S. Chakravarthy, G. Anderst-Kotsis, A.M. Tjoa, I. Khalil (Eds.), Database and Expert Systems Applications, in: Lecture Notes in Computer Science, vol.11707, 2019, pp. 351–361.

[41] H.V. Duong, N.T.H. Le, T. Tran, T.C. Truong, B. Le, P. Fournier-Viger, Efficient algorithms for mining closed and maximal high utility itemsets, Knowl.-Based Syst. 257 (2022) 109921.

[42] J.C. Lin, Y. Djenouri, G. Srivastava, P. Fournier-Viger, Efficient evolutionary computation model of closed high-utility itemset mining, Appl. Intell. 52 (9) (2022) 10604–10616.

[43] P. Fournier-Viger, J. Li, J.C. Lin, T. Truong-Chi, R.U. Kiran, Mining cost-effective patterns in event logs, Knowl.-Based Syst. 191 (2020) 105241.

[44] W. Gan, J.C. Lin, J. Zhang, H. Chao, H. Fujita, P.S. Yu, ProUM: Projection-based utility mining on sequence data, Inform. Sci. 513 (2020) 222–240.

[45] J. Wang, J. Huang, Y. Chen, On efficiently mining high utility sequential patterns, Knowl. Inf. Syst. 49 (2016) 597–627.

[46] J. Yin, Z. Zheng, L. Cao, USpan: An efficient algorithm for mining high utility sequential patterns, in: Proc. 18th ACM SIGKDD Intern. Conf. on Knowl. Discovery and Data Mining, 2012, pp. 660–668.